GSD

Руководство по интеграции

Интеграционный слой 1.1.0 / Исходное ядро 3.0.3

1. Назначение и граница поставки

Единый пакет объединяет неизменённый GSD Engine 3.0.3, документацию и интеграционный слой 1.1.0. Рабочая точка входа — `gsd.py` и `gsd_bridge/`. Именно этот путь обеспечивает проверку зависимостей до вызова компонентов, сборку контекста, регистрацию продуктов и защищённое от случайного повторного создания состояние.

Прямой вызов `engine/tools/` исполняет оригинальную реализацию без исправлений интеграционного слоя. Оригинал сохранён для воспроизводимости; он не является альтернативным исправленным входом. Предметные определения и исходные схемы берутся из `engine/data/`; собственные договоры соединения, контекста и хранения находятся в `contracts/`. Они не ослабляют исходные требования к результатам.

Пакет не содержит установленный Hermes, внешнюю модель, подключение Telegram, секреты или разрешённый исполнитель внешних действий. Готовность локального кода и готовность развёрнутой системы принимаются отдельно.

2. Архитектура и подключение

Интерфейс содержит 16 инструментов с едиными схемами для командной строки, MCP и модуля Hermes. Используйте один способ регистрации, не создавая дублирующий набор. MCP работает через стандартный ввод и вывод; реализованная редакция договора — 2025-06-18. Конкретная внешняя среда и её соответствие этому договору требуют собственной проверки.

`configure.py` создаёт отдельные настройки для Hermes, VS Code, Cursor, Claude Code, Codex или общего клиента. Скрипт не изменяет действующий профиль. Модуль Hermes запускает ядро в отдельном интерпретаторе, регистрирует инструменты, обработчики и команду `/gsd`. Обработчик перед моделью задаёт порядок работы, но не доказывает доставку всего корпуса.

Аналитическое ограничение модуля нельзя отключить параметром `enforce_analysis_only=false`: такая конфигурация отклоняется. Сам обработчик не защищает от его пропуска диспетчером среды и не контролирует автоматически чужие команды, дочерние процессы или сообщения. Полный запрет до эффекта должен обеспечиваться фактической средой.

Каталог содержит 1 948 определений: 152 процедуры, 37 протоколов, 31 отдельный результат и 1 728 исходных элементов. Восемь инструкций `SKILL.md` направляют работу с каталогом. Число определений не означает число самостоятельных программ.

3. Подготовка

Разделите каталог поставки, состояние, настройки и ключи. Используйте отдельное окружение Python и изолированный профиль. Локальная проверка выполнена на Python 3.13.5 под Linux; другие сочетания принимаются отдельно. Прямые зависимости записаны в `requirements.txt`, фактически использованные версии — в `requirements-tested.txt`.

Порядок команд приведён в `RUNBOOK.md`. Сначала проверьте происхождение и `gsd.py verify`, затем создайте окружение и только новое состояние. `init-state` не используется для продолжения существующей базы. Поставка размещается только для чтения; ключ состояния хранится отдельно и не попадает в архив.

Хеши подтверждают совпадение байтов, но не являются подписью автора. Защита SQLite и HMAC предполагает доверенную область ОС. Пользователь или процесс, имеющий доступ к ключу и базе, не изолирован от них этими механизмами.

4. Контекст и адресация

`gsd_context` принимает состав G/S/D, точные `entry_keys`, необязательные `procedure_ids`, профиль и предел байтов. Для процедур собираются полные исходные договоры и их зависимости. Чтение определения S из процедуры G или D не активирует S и не загружает его активную системную инструкцию.

Справочное основание, исполнение операции и принятие внешнего продукта — разные события. Манифест фиксирует владельца источника и `reference_only`. Краткий идентификатор разрешается только через зарегистрированный псевдоним или единственное точное совпадение; выбранный адрес записывается в `source_resolutions`. Неоднозначность приводит к отказу.

Инженерные операции требуют `profile=engineering`. Индекс содержит 305 разделов и подразделов, включая 69 разделов верхнего уровня. Консервативное замыкание доставляет весь неизменённый инженерный источник, включая обязательную основу §0, §3, §5, §46, §48, §50, §57, §58 и все его процедуры. Это полное, но не минимальное по объёму представление.

Предел `byte_limit` измеряется в байтах UTF-8, не в токенах; максимум — 8 МиБ. Переполнение означает отказ, а не сокращение норм. Манифест проверяется повторной сборкой, а не только сравнением его собственного хеша. Собранный файл, путь или сокращённый ответ инструмента не доказывают доставку модели. Полнота доставки и содержательное применение проверяются отдельно.

5. Конечные операции G/S/D

`gsd_execute` выполняет предусмотренные конечные операции. G принимает предоставленный набор либо вычисляет заданные вариации и комбинации; предметный метод `agent` требует настроенного исполнителя. S выполняет поддержанные проверки предсказаний и условные выводы. D сравнивает актуальные кандидаты на основании предъявленного строгого обзора.

Без исполнителя G зависимые S/D по умолчанию не вызываются, предоставленные кандидаты сохраняются, обязательство G остаётся открытым. Явное `generation_unavailable_policy=use_supplied_candidates` разрешает отдельную ветвь S/D на предоставленном поле. Такая ветвь не объявляет выполненным исходное требование GSD.

Ответ различает `invoked_components`, `executed_components`, `blocked_components`, `open_obligations`, происхождение кандидатов и выполнение новой генерации. Метод `supplied` не считается порождением новых кандидатов. Успешный вызов или локальное вычисление не устанавливает качество произвольного предметного результата.

6. Процедурный цикл и внешние зависимости

Для сохраняемой работы проверьте Case, создайте запуск, запросите следующую процедуру, исполните содержательную операцию, зарегистрируйте адресные результаты в Case и затем подтвердите попытку. `commit` не создаёт недостающий результат и не заменяет его проверку.

Все семь составов G, S, D, GS, GD, SD и GSD сохраняют применимые обязательства исходного диагностического контроллера. Они разделены на локальные и внешние; чужие обязательные процедуры не удаляются из плана. Это не семь полностью независимых автоматических процессов всего каталога.

При отсутствии локального владельца готовой обязательной процедуры возвращается `external_request`: её договор, входной отпечаток, тип результата и контрольные суммы контекста. Без внешнего исполнителя процесс остаётся незавершённым. Принятие внешнего продукта доступно только через отдельную команду среды и подписанную квитанцию; модельным инструментом она не является.

Проверяются доверенный исполнитель и его область, отзыв, подпись, срок, редакция случая, входы, контекст, контрольные суммы результатов, исходные зависимости и бюджет. Затем применяется исходный `commit` без сохранённого расширения локального состава. Подпись удостоверяет заявление исполнителя, а не истинность его содержания. Полный порядок — `PROCEDURAL_ROUTES.md`.

7. Продукты и передачи

`gsd_product_register` сохраняет исходный продукт без изменений внутри оболочки, назначает устойчивый идентификатор и целую версию, закрепляет входы и заявленное происхождение. Повтор одинакового содержимого не создаёт новую версию. Изменение требует ожидаемой предыдущей версии; история остаётся доступной, но не считается текущим результатом.

Собственный типизированный статус продукта сохраняется. При его отсутствии используется только `unreviewed`; описательная строка M не превращается в подтверждение. Регистрация не является независимой проверкой происхождения или истинности.

`gsd_product_handoff` собирает полный пакет из актуального продукта, а не только заголовок. Программный `ProductRegistry.deliver()` передаёт его через исходный локальный `HandoffChannel`. Сетевой транспорт, удостоверение получателя, распределённая атомарность и общий резерв среды не появляются от подготовки пакета. Договор приведён в `PRODUCTS.md`.

8. Состояние, повторы и общий расход случая

Первый `gsd_case_create` закрепляет `case_id` за семейством запусков. Повтор того же запроса возвращает прежний результат; новый `request_id` при том же случае не создаёт ещё один запуск. Продолжение идёт по сохранённому `run_id` через проверенный снимок и `restore()`.

Явный новый запуск разрешает оператор: указываются предшественник, актуальная редакция и основание. Накопленный расход семейства сохраняется, область бюджета не расширяется. Счётчики семьи проверяются до новой попытки. Это общий бюджет связанных процедурных запусков в одном хранилище, не общий учёт модели, Hermes или нескольких машин.

Снимки слоя 1.0.0 требуют явной миграции. Поддержан однозначный случай с одним старым запуском на `case_id`: проверка, резервная копия, сохранение того же исходного снимка и расходов. Несколько старых запусков одного случая требуют отдельного сверенного переноса; программа не выбирает историю произвольно. Порядок и ограничения — `LIFECYCLE.md`.

9. Полномочия и эксплуатация

Модельные инструменты работают только в аналитическом режиме. Лабораторный адаптер 1.2 не входит во входные архивы; интеграционный слой 1.1.0 не выдаётся за него. Недостающие внешние способности отражены в `ADAPTER_REQUIREMENTS.md`. При отсутствии доверенного допуска внешнее действие не исполняется.

Ключи внешних исполнителей и команды управления должны быть недоступны модельному процессу средствами ОС или отдельного сервиса. Флаг `--host-control` обозначает назначение команды, но сам не удостоверяет оператора. Подпись состояния не заменяет многопользовательскую авторизацию, шифрование или защиту от отката всей базы.

До включения рабочего профиля проверьте действительный путь блокировки действий, отказ обязательного обработчика, дочерние вызовы, сроки, повторы, бюджет, доставку контекста и восстановление. Неизвестный исход не повторяется автоматически под новым идентификатором. Инструкции эксплуатации и отката: `OPERATIONS.md` и `RUNBOOK.md`.

10. Приёмка

Фактические локальные результаты находятся в `verification/STATUS.md`, `verification/integration/`, `verification/upstream/` и `verification/remediation/`. Проверка сборки контекста 152 процедур не означает их содержательного исполнения. Подставной договор регистрации не означает работу установленного Hermes; проверка отдельного метода не означает Telegram от запроса до ответа.

Принимающая сторона заполняет `verification/acceptance_matrix.json` с версией среды, моделью, доказательствами и оставшимися ограничениями. Требования GSD-I01–GSD-I07 сопоставлены с кодом и адресными проверками в `verification/remediation/findings_matrix.json`. Развёртывание в действующем профиле разрешается отдельно после целевой приёмки.